公平组合游戏(博弈论)

公平组合游戏①(Impartial Combinatorial Game,ICG)是满足以下特征的一类问题：
(1) 有两个玩家，游戏规则对两人是公平的；
(2) 游戏的状态有限，能走的步数也有限；
(3) 两人轮流走步，一个玩家不能走步时，游戏结束；
(4) 游戏的局势不能区分玩家身份，像围棋这样有黑、白两方的游戏，就不属于此类问题。

ICG问题有一个特征：给定初始局势，并且指定先手玩家，如果双方都采取最优策略，那么获胜者就已经确定了。也就是说，ICG问题存在必胜策略。

本节讲解ICG问题的必胜策略。有关的知识点有P-position、N-position、Nim Game、Sprague-Grundy函数、Wythoff's Game等。ICG很早就得到了研究，如Nim Game问题，1902年，C. Bouton在一本著作中对它进行了分析；Sprague-Grundy函数，由数学家Grundy和Sprague在20世纪30年代分别独立发现。

Sprague-Grundy函数是本节最重要的内容。

> ① 在算法竞赛中常常称这类问题是“博弈论”问题，但实际上竞赛只涉及了博弈论中一个简单分支“公平组合游戏”。在普通的博弈论教材中一般没有公平组合游戏，在应用组合数学书中可能有，如《应用组合数学》(Alan Tucker著，速译，人民邮电出版社出版)第11章“图游戏”。

---

1 巴什游戏与P-position、N-position

巴什游戏是只有一堆石子的简单游戏，这种游戏的分析方法是P-position、N-position。

巴什游戏

问题描述：有n颗石子，甲先取，乙后取，每次可以拿$1\sim m$颗石子，轮流拿下去，拿到最后一颗的人获胜。
输入：n和m，$1 \le n$，$m \le 1000$ 。
输出：如果先拿的甲赢了，输出first；否则输出second。

问题分析

分析如下。

(1) $n \le m$ 时，由于一次最少拿一个，最多拿m个，甲可以一次拿完，先手赢。
(2) $n = m + 1$ 时，无论甲拿走多少个(1 ~ m个)，剩下的都多于1个且少于等于m个，乙都能一次拿走剩余的石子，后手取胜。

上面两种情况可以扩展为以下两种情况：
(1) 如果 $n\%(m+1) = 0$，即n是m+1的整数倍，那么不管甲拿多少，如k个，乙都拿 $m+1-k$ 个，使剩下的永远是m+1的整数倍，直到最后的m+1个，所以后拿的乙一定赢。
(2) 如果 $n\%(m+1) \ne 0$，即n不是m+1的整数倍，还有余数r，那么甲拿走r个，剩下的是m+1的倍数，这样就转移到了情况(1)，相当于甲乙互换，结果是甲赢。

在这个拿石子的游戏中，对于后拿的乙来说是很不利的，只有在 $n\%(m+1) = 0$ 的情况下乙才能赢，其他情况都是甲赢。

```
cin >> n >> m;
if(n % (m+1) == 0)
    printf("second\n");
else
    printf("first\n");
```

算法非常简单，若 $n\%(m+1)==0$，则先手败，否则先手胜。

取石头(洛谷P4018)

问题描述：两人玩游戏，共有n颗石头，两人每次只能取 p^k 个，p是素数，k是自然数，且 p^k 小于等于当前剩余石头数。谁取走最后的石头，谁就赢了。现在October先取，如果必胜，则输出"October wins!"；否则输出"Roy wins!"。
输入：输入一个正整数n，表示石头的个数。
输出：输出"October wins!"或"Roy wins!"。

问题分析

如果n是6的倍数，后手赢。如果n不是6的倍数，先手赢。
(1) 若 $n=1\sim5$，都存在 $p^k=n$，可以一次拿完，先手赢。当 $n=6$ 时，不存在 $p^k=6$，先手只能拿1～5个，后手把剩下的全拿走就赢了。
(2) n等于6的倍数时，一定不等于素数的k次方；证明：除了2以外的素数p都是奇数，而奇数乘奇数都是奇数，所以6的倍数都不是p的k次方；若p=2，由于6中存在因数3，6n也不是2的k次方。

如果n是6的倍数，先手无法拿6的倍数个，后手只要拿1～5个，把剩下的石头变成6的倍数，就赢了。

---

2. P-position、N-position与动态规划

上面对巴什游戏的解答虽然很好理解，但是稍作扩展就不容易了。例如，取石子的数量不是1～m的连续数字，而是只能在$\{a_1,a_2,...,a_k\}$中选。对于此类问题，需要一种通用的方法。

核心定义

- **P-position**：前一个玩家(Previous Player，即刚走过一步的玩家)的必胜位置，表示马上走下一步的先手必败。
- **N-position**：下一个玩家(Next Player)的先手必胜位置，表示马上走下一步的先手必胜。

设只能拿数量为{1,4}的石头，x是石头的数量，pos是对应的N/P-position。各石头数量对应的局势如下表：

位置计算(1)

x	0	1	2	3	4	5	6	7	8	9	10	11	12	13	...
pos	P	N	P	N	N	P	N	P	N	N	P	N	P	N	...

局势计算规则

(1) $x=0,1,2,3,4$时，$pos=P,N,P,N,N$。特别注意$x=0$，即没有石头的情况，可以看作下一个玩家(先手玩家)没有石头可拿，输了，$pos=P$。$x=1$时，先手玩家必赢，$pos=N$。$x=2$时，先手只能拿一个，后手拿剩下的一个，后手赢，所以先手必输，$pos=P$。
(2) $x=5$时，分两种情况：①如果先手玩家拿一个，退回到$x=5-1=4$的情况，此时后手玩家处于N；两种情况下，后手都赢了。所以$x=5$时，$pos=P$，即先手必输。

(3) $x=6$时，分别退回到$x=6-1=5$和$x=6-4=2$的情况，后手都处于P。两种情况下，后手都输了。所以$x=6$时，$pos=N$，先手必赢。
(4) $x=7$时，略。
(5) $x=8$时，退回到$x=8-1=7$，后手处于P；退回到$x=8-4=4$，后手处于N。在后手有输有赢的情况下，先手肯定选让对方必败的方案，所以$x=8$时，$pos=N$。

可以观察到，pos值是周期性变化的，周期为5。

下面再举一个例子，设只能拿数量为{1,3,4}的石头，验证结果如下表：

位置计算(2)

x	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
pos	P	N	P	N	N	N	N	P	N	P	N	N	N	N	P

可以看到，pos仍然是周期变化的，周期是7。

> 提示
> 上面的计算过程符合动态规划的思路。编程时可以用动态规划，也可以直接按照周期性变化规律计算。

尼姆游戏

巴什游戏只有一堆石子，如果扩展到多堆石头的复杂情况，就是尼姆游戏(Nim Game)。

尼姆游戏规则

有n堆石子，数量分别是$\{a_1,a_2,a_3,...,a_n\}$；两个玩家轮流拿石子，每次从任意一堆中拿走任意数量的石子；拿走最后一个石子的人获胜。

以3堆石子为例，简单情况的胜负是：

- {0,0,0}、{0,1,1}、{0,k,k}：先手必败。
- {1,1,1}、{1,1,2}、{1,1,3}：先手必胜。

核心胜负判定定理

对于任意的$\{a_1,a_2,a_3,...,a_n\}$，尼姆游戏有一个极其简单的胜负判断方法——**异或运算**。

定理 若$a_1\oplus a_2\oplus a_3\oplus\cdots\oplus a_n=0$，先手必败，记此时的状态为P-position；若$a_1\oplus a_2\oplus a_3\oplus\cdots\oplus a_n\neq 0$，先手必胜，记此时的状态为N-position。

例如，3堆石子数量分别是{5,7,9}，转换为二进制后进行异或运算，结果是

```
   0 1 0 1
   0 1 1 1
⊕ 1 0 0 1
------------
   1 0 1 1
```

异或运算的结果不等于0，先手必胜。

在数学中，二进制的异或运算，也可以看作统计每位上1的总个数的奇偶性。如果这一位上有偶数个1，那么这一位的计算结果为0；如果有奇数个1，计算结果为1。所以，尼姆游戏中的异或运算也被称为**Nim-sum**运算。

定理简单证明

(1) **必定能够从N-position转换到P-position。**也就是说，先手处于必胜点N-position时，可以拿走一些石子，让后手必败。具体方法：任选一堆石子，如第i堆，石头数量为k；对剩下的n-1堆进行异或运算，设结果为H；如果H<k，就把第i堆石子减少到H；这样操作之后，因为$H \oplus H = 0$，所以n堆石子的异或等于0。可以证明，总会存在这样的第i堆石子，而且可能有多种转化方案。
(2) **进入P-position后，下一个玩家，不管拿多少石子，都会转换到N-position。**因为任何一堆的数量变化都会使这一堆的二进制数至少有一位发生变化，导致异或运算的结果不等于0。
(3) **在游戏过程中，按上述步骤，在N-position和P-position之间交替转换，直到每堆石子都是0，即终止于P-position。**也就是说，这个玩家不管怎么拿石子，都必败。

上述证明过程也说明了玩家该如何进行游戏。下面给出尼姆游戏的例题

Being a good boy in spring festival(hdu 1850)

问题描述：桌子上有n堆扑克牌，每堆牌的数量分别为a_i，两人轮流进行，每步可以在任意一堆中取走任意张牌，桌子上的扑克全部取光，则游戏结束，最后一次取牌的人为胜者。问先手的人如果想赢，第1步有几种选择？
输入：n，表示牌的堆数；$a_i(i = 1 \sim n)$，分别表示每堆扑克的数量。
输出：如果先手能赢，输出他第1步可行的方案数，否则输出0。

主要代码

```
int sum = 0, ans = 0;
for(int i = 0; i < n; i++)  sum ^= a[i];     //sum为Nim-sum，ans为第1步可行方案数
if(sum == 0)
    cout << 0 << endl;                       //开局局面是P-position，先手必败
else {
    for(int i = 0; i < n; i++)               //开始局面是N-position，先手必胜
        if( (sum ^ a[i]) <= a[i] )
            ans++;
    cout << ans << endl;                      //计算第1步所有可能方案
}

```

代码中的`if((sum^a[i])<=a[i])`计算第1步的方案数，它利用了异或运算的原理：
设H等于除了a[i]之外其他所有数的异或，有
$sum = H \oplus a[i]$
$sum \oplus a[i] = H \oplus a[i] \oplus a[i] = H$
所以，代码中的`(sum^a[i])<=a[i]`就是$H <= a[i]$。把a[i]减少到H，就是一种可行的方案。